Day 3 — Refactoring to a Layer-Based Package Structure

Goal: Reorganize the flat Day 2 codebase into role-based packages *before* we add a Service layer and DTOs. **Result:** Five classes moved into `entity/`, `repository/`, `controller/`, `config/`; clean compile and green tests; nothing about the API's behaviour changed. **End-to-end touched:** Java packages → `package` declarations → cross-package `import`s → Spring component scanning → Maven build verification.

> This is the "doing" companion to concept-package-structure.md, which captures the *decision* (layer-based vs feature-based) and the interview trade-offs. Read that first for the **why**; this file is the **how** and the **what broke**.

---

1. Why refactor today (and not on Day 2)

On Day 2 every class lived flat in one package:

```
com.kuldeep.jobtrail/
├── JobtrailApplication.java
├── JobApplication.java
├── Status.java
├── JobApplicationRepository.java
├── JobApplicationController.java
└── DataSeeder.java
```

That was the *correct* choice then. Structure earns its keep the moment a "role bucket" holds more than one file. With one entity, one repo, one controller, packages would have been eight near-empty folders — **premature organization is its own technical debt.**

What changed: Day 3 introduces a **Service layer** and **DTOs**. That's the first time a role bucket will hold multiple files and the first time we'll want `package-private` visibility to hide internals. So the trigger fired — refactor first, build features second.

Rule to remember: *structure follows pressure.* You add packaging when flatness starts to hurt, not before.

---

2. The new structure

```
com.kuldeep.jobtrail/
├── JobtrailApplication.java        ← main; STAYS at the root
├── entity/
│   ├── JobApplication.java         ← @Entity
│   └── Status.java                 ← enum (part of the entity's data shape)
├── repository/
│   └── JobApplicationRepository.java
├── controller/
│   └── JobApplicationController.java
├── config/
│   └── DataSeeder.java             ← @Configuration
├── service/                        ← empty, waiting for JobApplicationService
└── dto/                            ← empty, waiting for request/response payloads
```

This is **layer-based** (a.k.a. by-role / horizontal) packaging: each package holds one *kind* of class. ~80% of Spring Boot codebases look like this, which is exactly why we chose it while learning — every tutorial and every interviewer maps onto it instantly.

Class	New package	Why
`JobtrailApplication`	root (`com.kuldeep.jobtrail`)	Convention-bound. `@SpringBootApplication` scans *from its own package downward*.
`JobApplication`	`entity/`	It's a JPA `@Entity`.
`Status`	`entity/`	The enum is part of the entity's data shape.
`JobApplicationRepository`	`repository/`	Spring Data JPA interface.
`JobApplicationController`	`controller/`	REST endpoint.
`DataSeeder`	`config/`	It's a `@Configuration` bean producer.

3. The mechanics — two things must stay in sync

Moving a Java file is never *just* moving a file. For each one, two things change in lockstep:

1. **The `package` declaration** at the top of the file must match its new directory under `src/main/java/`.
2. **Every `import` of a class that moved** must be added or updated — including in *other* files that reference the moved class.

Step 1 — move with `git mv` (preserve history)

```
git mv JobApplication.java           entity/JobApplication.java
git mv Status.java                   entity/Status.java
git mv JobApplicationRepository.java repository/JobApplicationRepository.java
git mv JobApplicationController.java controller/JobApplicationController.java
git mv DataSeeder.java               config/DataSeeder.java
```

Using `git mv` (rather than delete + create) lets Git record these as **renames**, so `git log --follow` keeps the file's history intact across the move.

Step 2 — fix the `package` line in each moved file

```
// entity/JobApplication.java
package com.kuldeep.jobtrail.entity;     // was: com.kuldeep.jobtrail
```

Step 3 — add the new cross-package imports

When everything lived in one package, classes saw each other **for free** (same-package access, no import needed). Splitting into packages breaks that — now they must import across the boundary.

File	New imports it needs
`repository/JobApplicationRepository.java`	`entity.JobApplication`
`controller/JobApplicationController.java`	`entity.JobApplication`, `repository.JobApplicationRepository`

File	New imports it needs
`config/DataSeeder.java`	`entity.JobApplication`, `entity.Status`, `repository.JobApplicationRepository`

Example — the repository after the move:

```
package com.kuldeep.jobtrail.repository;

import com.kuldeep.jobtrail.entity.JobApplication;   // <-- new: was same-package, now cross-packag
import org.springframework.data.jpa.repository.JpaRepository;

public interface JobApplicationRepository extends JpaRepository<JobApplication, Long> {
}
```

The compiler is your safety net here. Miss an import and the build fails *loudly* with "cannot find symbol" — there's no way to silently get this wrong.

---

4. The one thing that did *not* change: component scanning

A natural worry: *"if I move beans into sub-packages, will Spring still find them?"*

Yes — automatically, with zero config changes. Here's why:

`@SpringBootApplication` (on `JobtrailApplication`) bundles `@ComponentScan`, which defaults to scanning **the package the annotated class lives in, and all sub-packages.** Because `JobtrailApplication` sits at the root `com.kuldeep.jobtrail`, the new `com.kuldeep.jobtrail.entity`, `…repository`, `…controller`, `…config` are all *below* it — so they're discovered for free.

```
com.kuldeep.jobtrail          ← @SpringBootApplication scans from HERE...
├── entity/                   ←   ...downward, so all of these...
├── repository/               ←   ...are picked up automatically.
├── controller/
└── config/
```

Classic interview gotcha: if you ever move the main class *into* a sub-package (e.g. `com.kuldeep.jobtrail.app`), component scanning suddenly can't see its siblings, and beans

mysteriously stop loading. The fix is `@ComponentScan("com.kuldeep.jobtrail")` — but the real lesson is: *keep the `@SpringBootApplication` class at the root of your package tree.*

---

5. Verification — refactors must prove they changed nothing

A refactor is, by definition, a behaviour-preserving change. The way you *prove* that is the build:

```
./mvnw clean compile     # → BUILD SUCCESS (all packages/imports resolve)
./mvnw test              # → green (context loads, seeding runs under new structure)
```

The `contextLoads` test is doing real work here: it boots the entire Spring context. If any bean failed to be discovered after the move, or any import were wrong, this test would fail. It passing is our evidence that the layered structure is wired identically to the flat one.

---

6. Gotcha of the day — Maven offline mode and an incomplete cache

The first build attempts failed with errors that had **nothing to do with our code**:

```
Unable to load the mojo 'test' ... A required class is missing:
  org/apache/maven/plugin/surefire/SurefireReportParameters
...
maven-clean-plugin ... A required class was missing: org/codehaus/plexus/util/Os
```

Diagnosis: these are *Maven plugin* classes (surefire, clean-plugin, plexus-utils), not application classes. The local `~/.m2` cache was missing some plugin dependencies, and we'd run with the **`-o` (offline)** flag — so Maven couldn't download what was missing and failed.

Fix: drop `-o` and let Maven fetch the missing jars online. The build then went green.

Lesson: when a build error names `org.apache.maven.*` or `org.codehaus.plexus.*` classes, the problem is your **build tooling / cache**, not your source. Don't go hunting through your Java files for a bug that isn't there. And don't use `-o` until the cache is known-complete.

---

7. Interview anchors

1. **"I deliberately started flat and refactored when a role bucket got its second file."** — shows judgement about premature structure, not cargo-culting folders.
2. **Layer-based vs feature-based packaging** — I can name both, the trade-off (low coupling/tutorial-alignment vs high cohesion/easy deletion), and *why* I picked layer-based for a learning project. (Full version in concept-package-structure.md.)
3. **Component scan from the root package downward** — and the gotcha of moving the main class into a sub-package.
4. **Same-package vs cross-package access** — splitting packages forces explicit imports and *enables* `package-private` as a real encapsulation tool.
5. **A refactor is verified by the build, not by eyeballing** — `clean compile` + the context-load test prove behaviour is preserved.

---

8. What's next (rest of Day 3)

The structure is now ready for the features that triggered it:

- **`@Service` layer** — introduce `JobApplicationService`; the controller stops talking to the repository directly and calls the service. Business logic gets a home.
- **DTOs** — add a `JobApplicationResponse` (and later a request DTO) so the API stops leaking the raw JPA entity. Decouples the wire format from the database schema.
- **Write endpoints** — `POST /api/v1/applications` and `GET /api/v1/applications/{id}` (introduces `@RequestBody`, `@PathVariable`, and `Optional` handling) to give the service real work to do.

Files changed today

```
src/main/java/com/kuldeep/jobtrail/
├── JobtrailApplication.java          (unchanged — stays at root)
├── entity/
│   ├── JobApplication.java           ← moved + package/imports
│   └── Status.java                   ← moved + package
├── repository/
│   └── JobApplicationRepository.java ← moved + package/imports
```

```
├── controller/
│   └── JobApplicationController.java ← moved + package/imports
└── config/
    └── DataSeeder.java               ← moved + package/imports
docs/
├── notes/day-3.md                    ← this file
├── notes/concept-package-structure.md← the decision doc (the "why")
└── qna/day-3.md                      ← Q&A learning log
```